



Microduck 硬件架构拆解

\$399 的开源双足机器鸭，15 只舵机和一块 60 元级主控板，钱都花在哪？一根线怎么管 16 个设备？

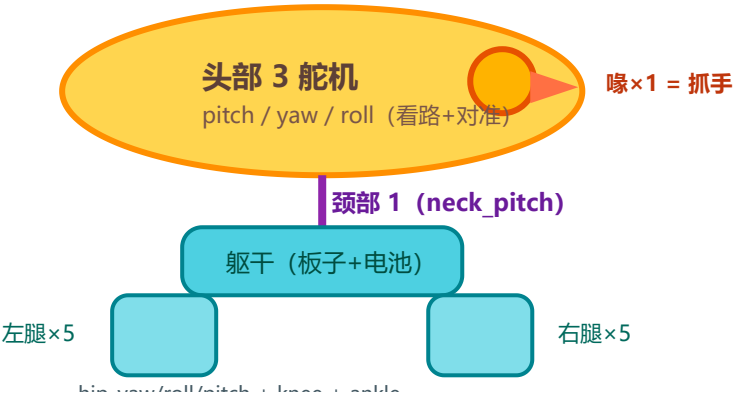
拾遗补充篇 · 基于 pollen-robotics/microduck v0.15.1 源码逐行验证 + 硬件 teardown 交叉核对 · 2026-10-02

📖 **姊妹篇说明**：本项目（Pollen Robotics / Hugging Face 出品，前身 Open Duck Mini）在拾遗流水线中另有一篇通用《小白指南》（软件栈：Rust 多守护进程 + RL 策略 + MuJoCo 仿真）。**本篇专拆硬件**——舵机怎么分、总线怎么排、算力够不够、成本合不合，全部结论都有源码文件级出处。

① 疑问即目录

Q1 15 个舵机是怎么分配的？一只鸭子哪来 15 个关节？

别被"鸭子"骗了——它是**标准小型双足布局 + 一个鸟喙彩蛋**。我们直接解析了仓库内嵌的运动模型 `kinematics/assets/alpha/robot_walk.xml`，**14 个行走关节 + 1 个喙 = 15 个 XL330 舵机**，一个不多一个不少：



部位	舵机数	关节名 (源码实测)	作用
头部	3	head_pitch / head_yaw / head_roll	摄像头指向、看路
颈部	1	neck_pitch	俯仰补视野
左腿	5	hip_yaw / hip_roll / hip_pitch / knee / ankle	行走 RL 策略执行
右腿	5	同上 (right_前缀)	行走 RL 策略执行
喙	1	beak (行走模型不含)	抓取 + 唱歌对口型

🔍 出处：MJCF 里逐个 joint 摘出来的 (`<joint name="left_ankle"> ...`)；喙舵机不在行走 MJCF 里，但 `robotd/src/theremin.rs`、`chorale.rs` 全是它的驱动逻辑——手挡在喙前变音符，唱歌时喙跟着元音开合。

Q2 为什么选 Dynamixel XL330？和几块钱的普通舵机差在哪？

普通 SG90 舵机是"哑巴"：给 PWM 波就转，**不知道自己在哪**，关节角度要外接电位器/编码器才知道。XL330 是**智能舵机：内置磁编码器 + 单片机 + 串口协议栈**，主机发"转到位置 2048"它自己闭环到位，还能汇报当前位置/速度/温度/负载/是否堵转。做双足步行，每个关节必须 50Hz 实时反馈——哑巴舵机方案的光是传感线就是一场噩梦。

XL330 关键参数 (Robotis 官方数据)：扭矩约 0.5 N·m 量级、重约 22g、磁编码 12bit (0~4095)、TTL 串口半双工、支持 Protocol 2.0；固件 **v46+** 支持 **fast sync read (指令 0x8A)**——Microduck 源码里明确依赖这一特性 (`robotd-params/src/lib.rs:1798`)。

⚠️ 代价：一只零售 **\$27.49** (XL330-M288-T)，15 只 = **\$412.35**，**已经超过整机定价 \$399**——这也是 Q9 的伏笔。

Q3 16 个设备怎么共用一根线不吵架？——一根 UART 菊花链

架构文档原话：**"15 servos + IMU on one UART"**。Dynamixel 总线是**半双工串口 + ID 寻址**：所有设备并联在一根三线（电源/地/数据）总线上，每个设备一个唯一 ID，只有被点到名的才应答。类比：这不是 16 条私聊热线，而是**一条对讲机频道**——报 ID 喊话，被叫者回答，其余闭嘴监听。



省时间的杀招：普通 sync read 是逐个点名——16 个设备 = 16 次应答 = 16 个帧头 + 16 次总线换向。

Microduck 用 fast sync read (0x8A)：**一次广播，16 个设备把应答接续追加到同一个状态包里**。源码注释原话："drops fifteen packet headers and fifteen bus turnarounds from the tick" (duck-control/src/bus.rs:7-9)。50Hz 控制环每 20ms 一拍，总线时间是硬约束。

```
# 每个 20ms 控制拍 (robotd 主循环)：
sync_read(0x8A) → 16 设备应答接力 → IMU + 15×位置/速度
计算 RL 策略 (ONNX, RK3566) → 新目标位置
sync_write(0x83) → 15 个舵机一次性收到各自目标
```

Q4 大脑 RK3566 有多强？能在鸭子上跑大模型吗？

主控是 **Rockchip RK3566 (Radxa Zero 3 板, 1GB RAM / 32GB 存储)**——4×A55@1.8GHz 的入门级 Linux 板，还有个 1 TOPS NPU。能跑什么？源码和 FAQ 说得很诚实：

- **能跑**：步行 RL 策略 (MLP 小网络，导出 ONNX, 50Hz 推理)、小目标检测器 (NPU 上)、WebRTC 音视频、7 个守护进程。
- **不能跑**：任何真正的 VLM/大模型。FAQ 第一条就是答案：**"I want to run a model that is too heavy for the board → Run it somewhere else and send it the camera"**。

重活外包给 Hugging Face Space (GPU 容器)：机器人用 WebRTC 推 H.264 视频流出去，云端看视频做决策，控制指令走同一条会话回来。**这正是知识库 [[控制面与数据面]] 思想的机器人版**：板载控制面 (50Hz 安全环) 永远本地、永远在线；数据面 (重模型推理) 按需上云。NAT 打不通还有 Cloudflare 短时凭证中继兜底 (10GB/月免费流量)。

🔗 出处：docs/faq.md (Radxa Zero 3、NPU 只带小检测器、WebRTC 方案)；robotd/src/soc.rs (soc-thermal / gpu-thermal 双热区读取，坐实 Zero 3 板型)。

Q5 它怎么知道要摔了？传感器都藏在哪儿？

三套感知，各管一段：

传感器	规格	作用
IMU ×2	总线 ID 表第一位（数据最先到）	躯干姿态/角速度——RL 步行策略的眼睛，摔没摔、往哪歪全靠它，50Hz 与舵机同拍
ToF 深度矩阵	ST VL53L8CX, 8×8 = 64 区测距	头“眼睛”下方的深度雷区：避障、检测手（特雷门琴）
摄像头 + 麦克风	经 mediad 编码 H.264	远程视觉 / 语音交互

8×8 矩阵怎么用？ 源码注释很克制：“no reprojection”——只给原始 64 区距离，不反投影成 3D 点云。够用的场景（前面有没有障碍/手）用矩阵原始值就行，省掉全部几何换算。**这不是扫描式激光雷达，是 64 像素的“测距相机”**——teardown 帖叫它“LiDAR”是营销说法，源码管它叫 depth matrix。

VL53L8CX 视野：8×8 区，每区一个距离值

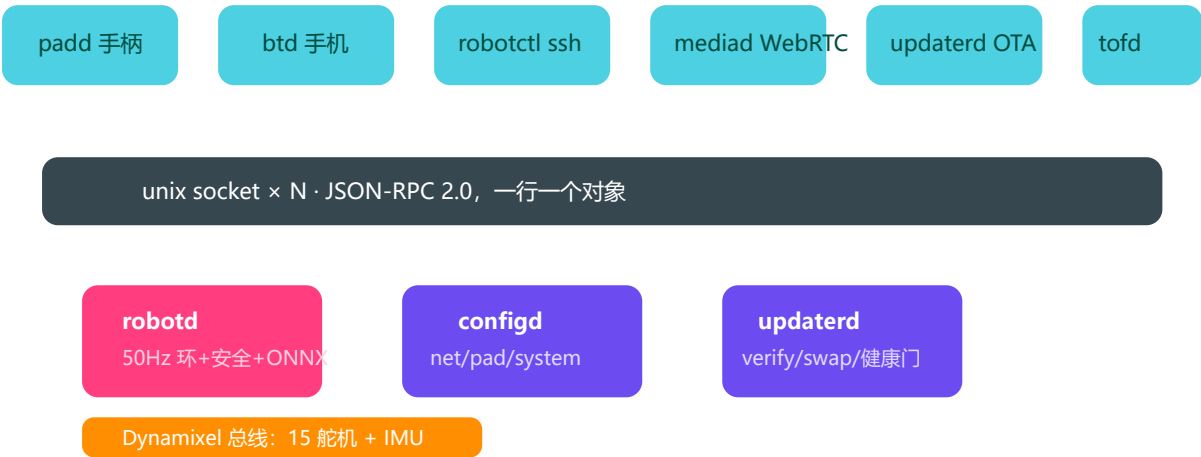


红=近（手！）
蓝=远
→ 触发音符

🔍 出处：tof/src/lib.rs ("The head ToF sensor: an 8×8 matrix of distances" / "no reprojection") ; tof/vendor/vl53l8cx/ (ST 官方驱动直接 vendor 进仓库) ; robotd/src/theremin.rs ("The ToF theremin: a hand's distance in front of the beak becomes a note") 。

Q6 一块小板子跑 7 个守护进程？为什么不多合一？

架构文档原话：**"Seven daemons on one board, talking over unix sockets. One of them drives the robot; three of the others exist so that the first one can be broken without the board becoming unreachable"**——三个进程的存在理由就是"robotd 挂了板子还能救回来"。



进程切分逻辑 = 按"崩溃后果"切，不按功能切：robotd 管 50Hz 实时环（最可能出 bug 的）；configd/updaterd 与它隔离——就算 robotd 死循环，你还能 ssh 进去回滚固件。tofd 最特别：**"publishes, answers nothing"**（只发不收的传感器守护进程），把 8×8 深度矩阵广播到 /run/tofd/tof.sock，谁要谁读。

关联知识库：这与 [[MCP模型上下文协议]] 的"小入口+能力分体"、[[JSON-RPC与stdio传输]] 的"一行一个对象"同构——unix socket 上的 JSON-RPC 就是机器人界的 stdio 协议纪律。

Q7 \$399 卖一台亏一台？——BOM 账本拆给你看

teardown 实测口径：XL330-M288-T 官方零售 \$27.49/只。15 只 × \$27.49 = \$412.35 > 整机 \$399——即只算舵机，官方定价已经在亏钱。我们写了三情景计算器（实验 3，真实运行）：

情景	15×XL330	主板	IMU×2	ToF	电池(NP-F550)	3D打印结构	其他	合计	vs \$399
悲观/单件零售	\$412.35	28	16	45	15	12	18	\$546	-147
中性/小批量	\$315.00	22	14	35	12	10	15	\$423	-24
乐观/大批量	\$262.50	18	12	30	10	8	12	\$353	+46

商业模式就写在账本里：软件（含 RL 训练、仿真、WebRTC 栈）全部 Apache-2.0 开源；硬件 BOM 与结构件不开源。Hugging Face 卖的不是鸭子，是"开源生态的标准硬件锚点"——类似 early Raspberry Pi 的打法：硬件贴近成本卖，赚生态位。前身项目 Open Duck Mini 本来就是社区 DIY 路线（自己买舵机自己打印），Microduck 是它的"官方组装版"。

Q8 我能自己造一只吗？50Hz 控制环小白在家怎么验证？

分层回答：**仿真层 100% 可以**（姊妹篇已实测 MuJoCo + 仓库内嵌 MJCF，零硬件跑通）；**硬件层部分可以**——Open Duck Mini 社区路线就是自购 XL330 + 自打印结构件，但整机结构件/内部件 Microduck 不开源，**复刻一台“完全体”做不到，复刻“控制架构”完全做得到**：一只 XL330 (~¥200) + USB-TTL 线 + 5V 电源，就能真实跑通本文实验 2 的协议层代码（把假总线换成真串口）。

⚠ 本文诚实记录：**本机没有鸭子**。实验 2 是协议层正确性验证（构造→CRC→回读→篡改检测），不是实机总线通信。别把“跑通了 CRC”当成“驱动过舵机”。

② 动手实验（无硬件轻量路线，真实运行记录）

实验脚本：microduck\exercise\hardware\duck_hardware_lab.py（纯标准库，零依赖）。三段实验对应三个疑问：Q1 的 15-DOF 地图、Q3 的总线协议、Q7 的成本账本。

步骤 1：从源码提取 15-DOF 硬件地图

操作：PYTHONUTF8=1 python microduck/exercise/hardware/duck_hardware_lab.py（脚本内解析 kinematics/assets/alpha/robot_walk.xml）

目的：验证“15 个舵机 = 头3+颈1+腿5×2+喙1”不是媒体转述，而是能从源码复算的事实。

✅ **真实输出（节选）：**

```
左腿：5 个舵机  <- left_hip_yaw, left_hip_roll, left_hip_pitch, left_knee, left_ankle
右腿：5 个舵机  <- right_hip_yaw, right_hip_roll, right_hip_pitch, right_knee, right_ankle
喙（抓取/表达，第 15 舵机）：1 个  <- beak（行走 MJCF 不含，见 robotd theremin/chorale 模块）
合计：15 个 XL330 舵机  [15 = 头3 + 颈1 + 腿5x2 + 喙1]  PASS
```

步骤 2：Dynamixel Protocol 2.0 协议层验证（CRC + Sync Write）

操作：同上脚本实验 2 段——CRC-16 (poly 0x8005) 写两版（逐位 LFSR / 256 项查表）交叉验证，再用标准目录向量校准，最后构造 16 设备广播包并回读解码 + 篡改测试。

目的：不插一只舵机，也能证明“我们写的总线驱动层在协议上是对的”——这就是无硬件路线能做到的诚实上限。

✅ **真实输出：**

```
2a. CRC 双实现交叉验证 + 标准向量：bitwise=0xfee8 table=0xfee8 标准=0xfee8  PASS
2b. Sync Write(0x83) 广播包：16 个设备，包长 122 字节  PASS
2c. 回读解码：ID=0xfe 指令=0x83 参数逐字节一致  PASS
2d. 篡改 1 个 CRC 位必须被拒绝（总线完整性）  PASS
```

怎么验证 CRC 没写错：“123456789”的 CRC = 0xFEE8 是 **CRC-16/BUYPASS** 的国际标准校验值（RevEng CRC 目录收录），双实现 + 标准向量三方一致才算数。

步骤 3: BOM 三情景计算器

操作: 同上脚本实验 3 段。

✅ 真实输出:

```
悲观/单件零售 15×XL330=$412.35 ... 合计 $546.35 毛利空间 -147
乐观/大批量 15×XL330=$262.50 ... 合计 $352.50 毛利空间 +46
核心断言: 零售价 15 只舵机=$412.35 > $399, 且 悲观>$399>乐观 PASS
```

📖 过程如实记录: 第一版断言写成"所有情景在 \$200~480 区间", 悲观情景 \$546 触发 FAIL——其实悲观超预算恰恰是结论本身。修正断言语义 (零售>399>批量) 后 3/3 PASS。教训: **先想清楚"什么结果算验证成功", 再写断言。**

③ 能力清单: 这套硬件架构教会我们什么

- **智能舵机选型判断:** 什么时候值得为"内置编码器+协议"付 30 倍价钱 (要关节级反馈的腿式机器人 vs 只要转个角度的云台)
- **单总线多设备架构:** ID 寻址 + fast sync read, 把"16 设备轮询"压成"1 次接力应答"——同样思路可用于任何串口设备群
- **SoC 边界感:** 1 TOPS NPU 的板子该跑什么 (小 ONNX)、该外包什么 (VLM 走 WebRTC) ——**算力不够不是问题, 边界划错才是**
- **按崩溃后果切进程:** 实时环与救砖通道物理隔离, 比"一个大单体"和"微服务全家桶"都更适合嵌入式
- **硬件成本结构直觉:** 执行器占腿式机器人 BOM 一半以上; 开源硬件项目的定价=生态位投资

拾遗 task 6083ef55 · 硬件架构拆解补充篇 (姊妹篇: Microduck 小白指南·软件栈) · 数据来源: pollen-robotics/microduck v0.15.1 源码逐行核对 + Robotopian BOM teardown + Pollen Robotics 官网 · 实验真实运行于 2026-10-02